

metrics for search analysis

CSCI
270

from the
wikipedia
article on
depth-first
search

Properties [\[edit \]](#)

The **time** and **space** analysis of DFS differs according to its application area. In theoretical computer science, DFS is typically used to traverse an entire graph, and takes time $O(|V| + |E|)$,^[4] linear in the size of the graph. In these applications it also uses space $O(|V|)$ in the worst case to store the stack of vertices on the current search path as well as the set of already-visited vertices. Thus, in this setting, the time and space bounds are the same as for **breadth-first search** and the choice of which of these two algorithms to use depends less on their complexity and more on the different

the sliding tile puzzle



slide the tiles
to put them in
numerical order

"3-puzzle"

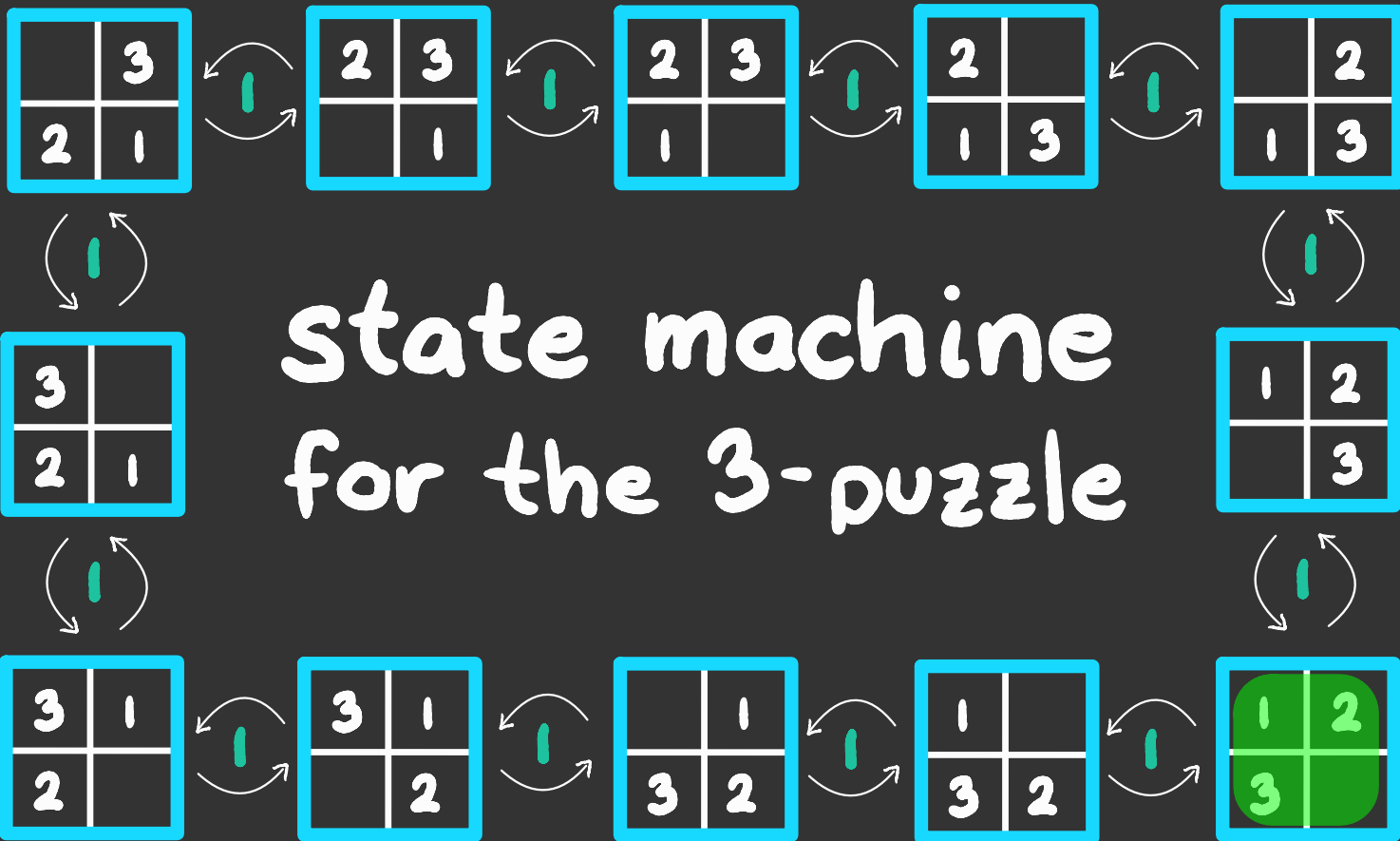
1	2
3	

"8-puzzle"

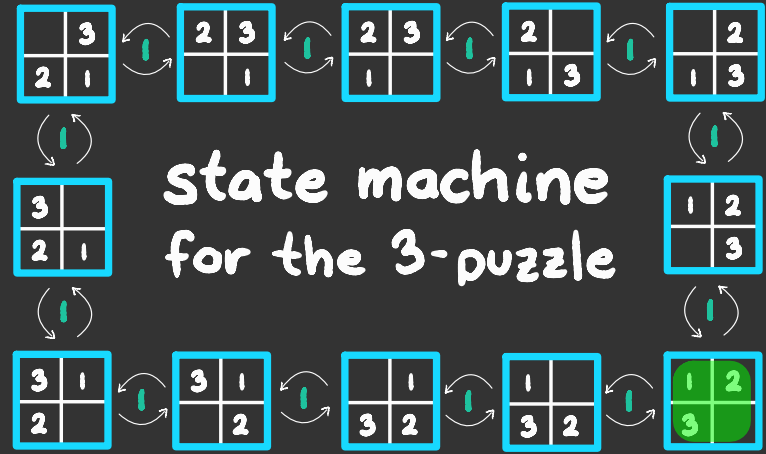
1	2	3
4	5	6
7	8	

"15-puzzle"

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	



in general, the
state machine for
the n -puzzle has

$$\frac{(n+1)!}{2} \text{ states}$$


"3-puzzle"

1	2
3	

$\frac{4!}{2}$
states

"8-puzzle"

1	2	3
4	5	6
7	8	

$\frac{9!}{2}$
states

"15-puzzle"

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	

$\frac{16!}{2}$
states

"3-puzzle"

1	2
3	

12

states

"8-puzzle"

1	2	3
4	5	6
7	8	

$\frac{9!}{2}$

states

"15-puzzle"

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	

$\frac{16!}{2}$

states

"3-puzzle"

1	2
3	

12

states

"8-puzzle"

1	2	3
4	5	6
7	8	

181440

states

"15-puzzle"

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	

$\frac{16!}{2}$

states

"3-puzzle"

1	2
3	

12

states

"8-puzzle"

1	2	3
4	5	6
7	8	

181440

states

"15-puzzle"

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	

10461394944000

states

"15-puzzle"

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	

10461394944000
states

so the runtime of dfs is linear
... in 10 trillion states

Properties [\[edit \]](#)

The **time** and **space** analysis of DFS differs according to its application area. In theoretical computer science, DFS is typically used to traverse an entire graph, and takes time $O(|V| + |E|)$,^[4] linear in the size of the graph. In these applications it also uses space $O(|V|)$ in the worst case to store the stack of vertices on the current search path as well as the set of already-visited vertices. Thus, in this setting, the time and space bounds are the same as for **breadth-first search** and the choice of which of these two algorithms to use depends less on their complexity and more on the different

and what about the
36-puzzle?



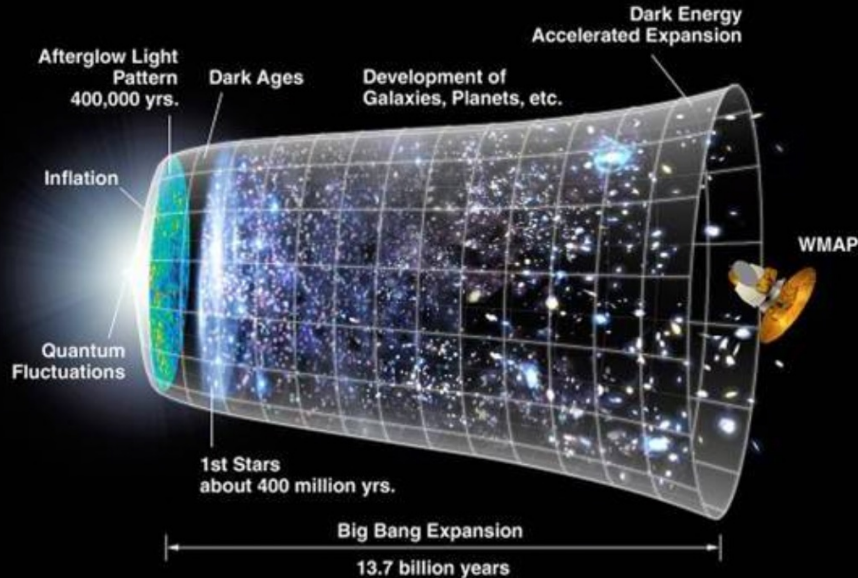
its state machine has about

1.86×10^{41}

states



the 64-puzzle has more states than
atoms in the observable universe



for problems like these, the number of states provides little insight into the performance of search algorithms

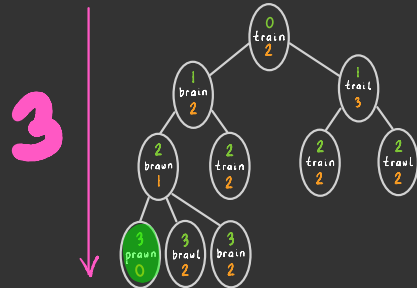
Properties [\[edit \]](#)

The time and space analysis of DFS differs according to its application area. In theoretical computer science, DFS is typically used to traverse an entire graph, and takes time $O(|V| + |E|)$,^[4] linear in the size of the graph. In these applications it also uses space $O(|V|)$ in the worst case to store the stack of vertices on the current search path as well as the set of already-visited vertices. Thus, in this setting, the time and space bounds are the same as for breadth-first search and the choice of which of these two algorithms to use depends less on their complexity and more on the different

but there are other properties of state machines that more effectively characterize the performance of search algorithms

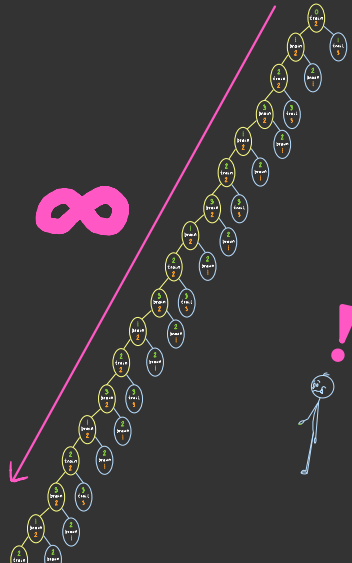
solution depth

the length of the
shortest search path
that ends at a
final state



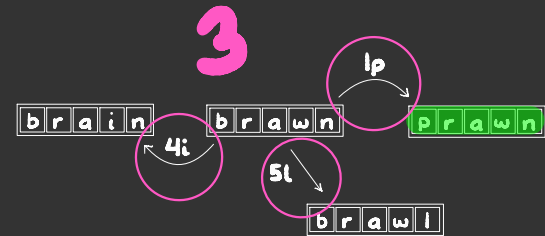
maximum depth

the length of the
longest search path



branching factor

the maximum number of
successors
of a state



suppose this is the initial
state of a 15-puzzle

1	2	3	4
5	6		8
9	10	7	12
13	14	11	15

solution depth

maximum depth

branching factor

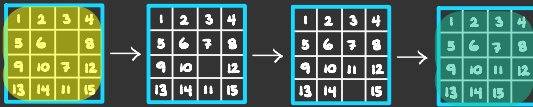


suppose this is the initial state of a 15-puzzle

1	2	3	4
5	6		8
9	10	7	12
13	14	11	15

solution depth

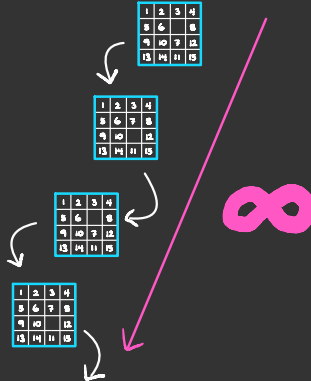
the length of the shortest search path that ends at a final state



3

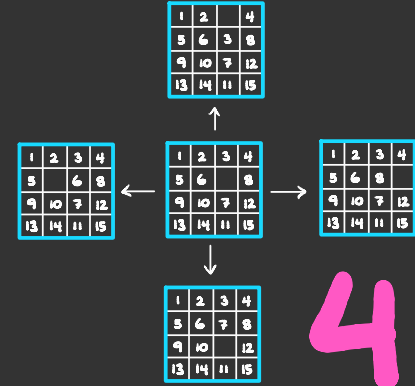
maximum depth

the length of the longest search path



branching factor

the maximum number of successors of a state



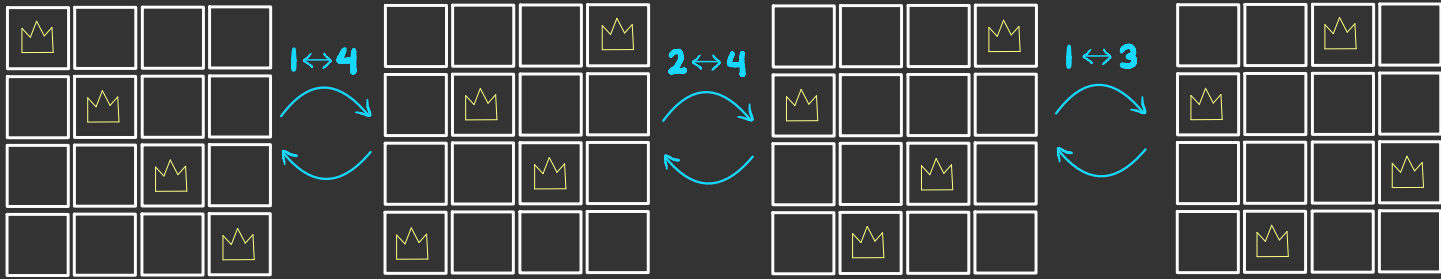
recall that n-queens had

multiple

state machine formulations

formulation one

swap rows until you reach a solution



solution depth



maximum depth

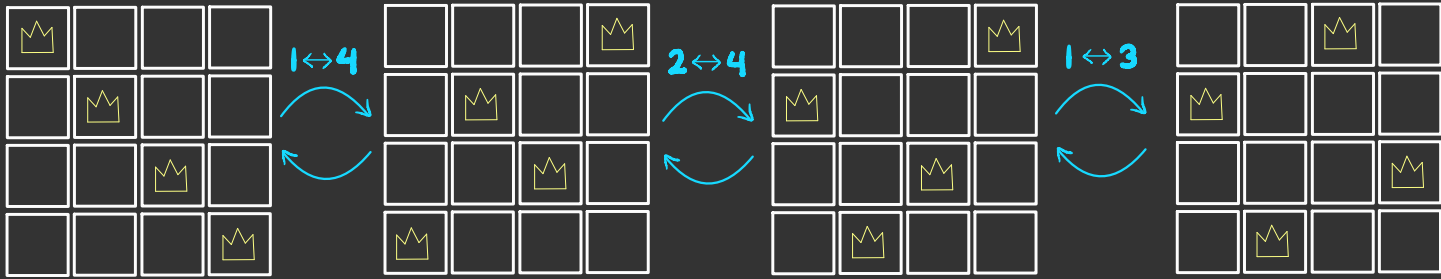


branching factor



formulation one

swap rows until you reach a solution



solution depth

$< n$

(if a solution exists)

maximum depth

∞

(state machine has reachable cycles)

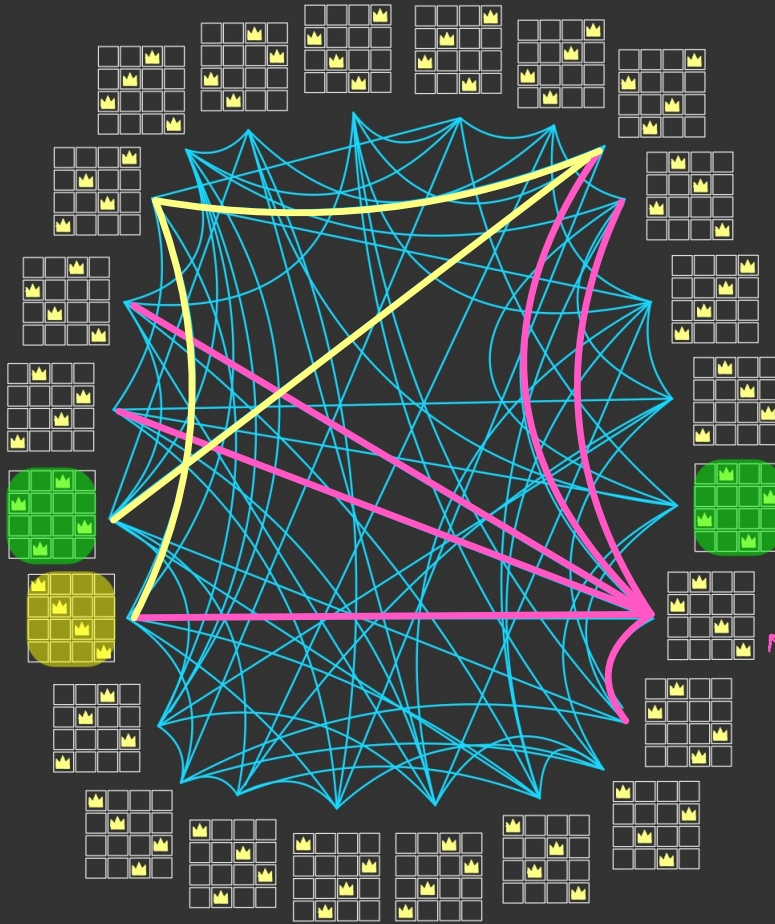
branching factor

$\binom{n}{2}$

solution depth

$< n$

(if a solution exists)



maximum depth

∞

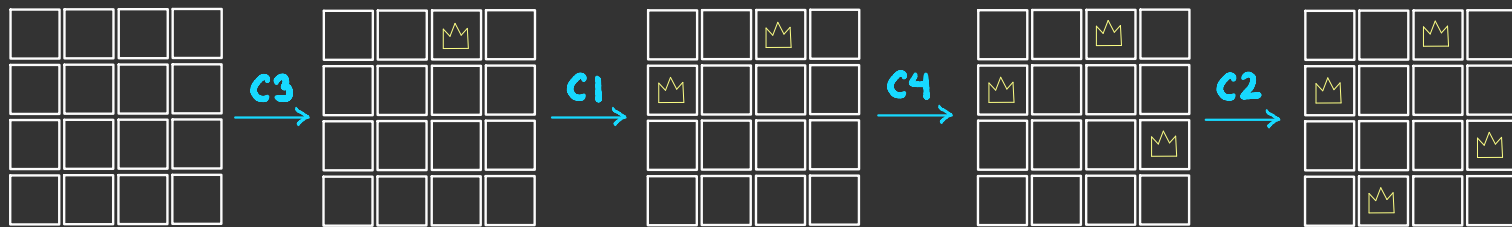
(state machine
has reachable cycles)

branching factor

$\binom{n}{2}$

formulation two

add queens until you reach a solution



solution depth



maximum depth

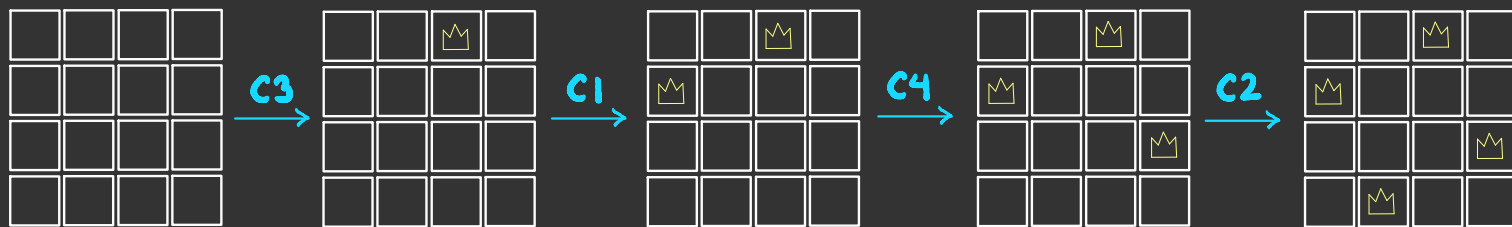


branching factor



formulation two

add queens until you reach a solution



solution depth

n

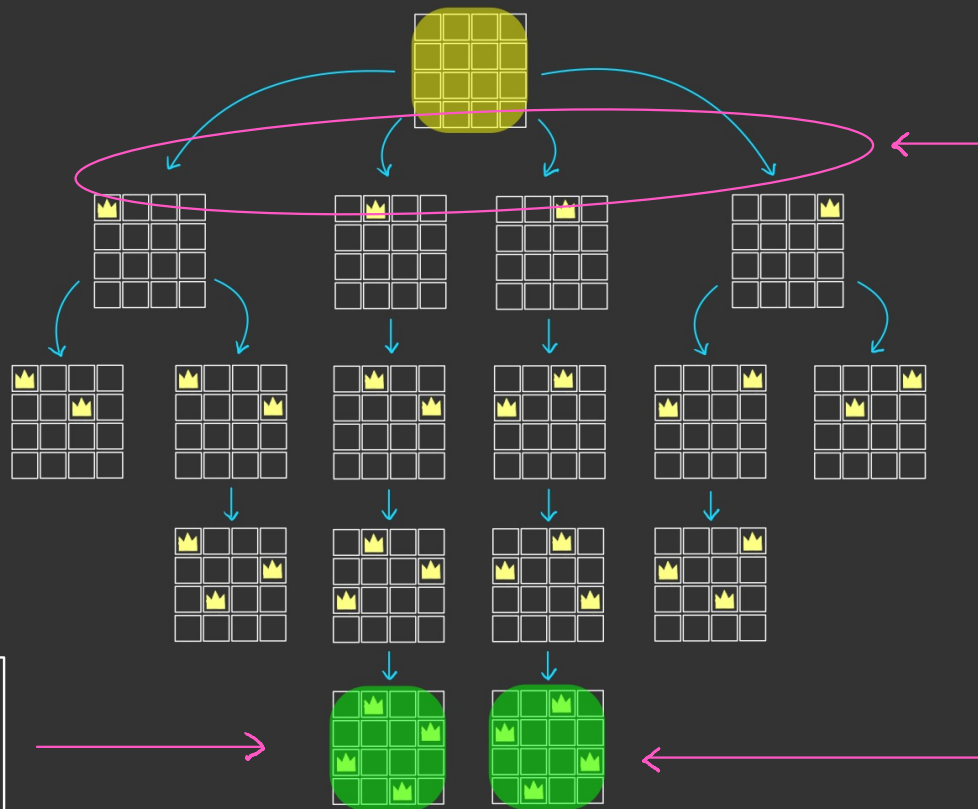
maximum depth

n

branching factor

n

(if a solution exists)



branching factor

n

maximum depth

n

solution depth

n

(if a solution exists)